

JAVA COLLECTION FRAMEWORK

Important Methods by Class – Assignment

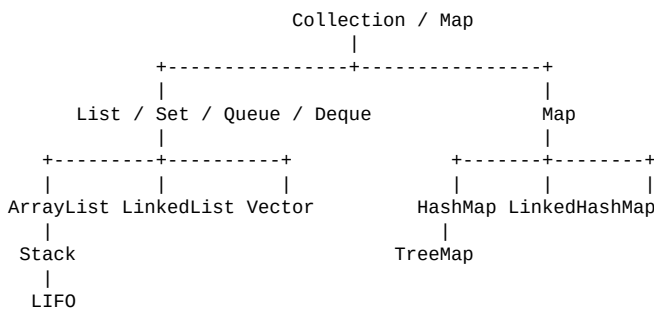
1. AIM

To study and demonstrate important methods of commonly used Java Collection Framework classes such as ArrayList, LinkedList, Vector, Stack, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, HashMap, LinkedHashMap, TreeMap and Hashtable.

2. DESCRIPTION

The Java Collection Framework provides ready-to-use classes for storing and processing groups of objects. List classes maintain ordered elements, Set classes store unique elements, Queue and Deque classes support queue operations, and Map classes store key-value pairs. Different implementations provide features such as insertion order, sorting, priority-based retrieval and synchronization.

3. UML DIAGRAM



Set implementations:

HashSet -> LinkedHashSet -> TreeSet

Queue/Deque:

PriorityQueue

ArrayDeque

4. SOURCE CODE WITH COMMENTS

```
import java.util.*;

public class CollectionClassesDemo {

    public static void main(String[] args) {

        // 1. ArrayList - ordered list and allows duplicates
        ArrayList<String> list = new ArrayList<>();
        list.add("B");
        list.add("A");
        list.add("B");
        list.add(1, "C");
        System.out.println("ArrayList: " + list);
        System.out.println("get(1): " + list.get(1));
        list.set(1, "D"); // Replace element
        System.out.println("After set: " + list);
        System.out.println("indexOf B: " + list.indexOf("B"));
        System.out.println("Size: " + list.size());

        // 2. LinkedList - supports list and queue operations
        LinkedList<String> linked = new LinkedList<>();
        linked.add("B");
        linked.addFirst("A");
        linked.addLast("C");
        System.out.println("\nLinkedList: " + linked);
        System.out.println("getFirst(): " + linked.getFirst());
        System.out.println("getLast(): " + linked.getLast());
        System.out.println("poll(): " + linked.poll());
        System.out.println("After poll: " + linked);

        // 3. Vector - synchronized resizable array
        Vector<String> vector = new Vector<>();
        vector.add("A");
        vector.addElement("B");
```

```

vector.add("C");
System.out.println("\nVector: " + vector);
System.out.println("Size: " + vector.size());
System.out.println("Capacity: " + vector.capacity());

// 4. Stack - LIFO
Stack<String> stack = new Stack<>();
stack.push("A");
stack.push("B");
stack.push("C");
System.out.println("\nStack: " + stack);
System.out.println("peek(): " + stack.peek());
System.out.println("pop(): " + stack.pop());
System.out.println("After pop: " + stack);

// 5. HashSet - unique elements
HashSet<String> set = new HashSet<>();
set.add("A");
set.add("B");
set.add("A"); // Duplicate ignored
System.out.println("\nHashSet: " + set);
System.out.println("Contains A: " + set.contains("A"));
System.out.println("Size: " + set.size());

// 6. LinkedHashSet - maintains insertion order
LinkedHashSet<String> linkedSet = new LinkedHashSet<>();
linkedSet.add("C");
linkedSet.add("A");
linkedSet.add("B");
System.out.println("\nLinkedHashSet: " + linkedSet);

// 7. TreeSet - sorted and unique
TreeSet<Integer> treeSet = new TreeSet<>();
treeSet.add(30);
treeSet.add(10);
treeSet.add(20);
treeSet.add(40);
System.out.println("\nTreeSet: " + treeSet);
System.out.println("first(): " + treeSet.first());
System.out.println("last(): " + treeSet.last());
System.out.println("higher(20): " + treeSet.higher(20));
System.out.println("lower(20): " + treeSet.lower(20));
System.out.println("ceiling(25): " + treeSet.ceiling(25));
System.out.println("floor(25): " + treeSet.floor(25));

// 8. PriorityQueue - priority-based queue
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(30);
pq.add(10);
pq.add(20);
System.out.println("\nPriorityQueue: " + pq);
System.out.println("peek(): " + pq.peek());
System.out.println("poll(): " + pq.poll());
System.out.println("After poll: " + pq);

// 9. ArrayDeque - double-ended queue
ArrayDeque<String> deque = new ArrayDeque<>();
deque.addFirst("B");
deque.addLast("C");
deque.addFirst("A");
System.out.println("\nArrayDeque: " + deque);
System.out.println("peekFirst(): " + deque.peekFirst());
System.out.println("peekLast(): " + deque.peekLast());
System.out.println("pollFirst(): " + deque.pollFirst());
System.out.println("After pollFirst: " + deque);

// 10. HashMap - key-value pairs
HashMap<Integer, String> map = new HashMap<>();
map.put(101, "A");
map.put(102, "B");
map.put(103, "C");
System.out.println("\nHashMap: " + map);
System.out.println("get(102): " + map.get(102));
System.out.println("containsKey(101): " + map.containsKey(101));
System.out.println("Keys: " + map.keySet());
System.out.println("Values: " + map.values());

// 11. LinkedHashMap - maintains insertion order
LinkedHashMap<Integer, String> linkedMap = new LinkedHashMap<>();
linkedMap.put(101, "A");
linkedMap.put(102, "B");
linkedMap.put(103, "C");
System.out.println("\nLinkedHashMap: " + linkedMap);
System.out.println("entrySet(): " + linkedMap.entrySet());

// 12. TreeMap - sorted keys

```

```

TreeMap<Integer, String> treeMap = new TreeMap<>();
treeMap.put(30, "C");
treeMap.put(10, "A");
treeMap.put(20, "B");
treeMap.put(40, "D");
System.out.println("\nTreeMap: " + treeMap);
System.out.println("firstKey(): " + treeMap.firstKey());
System.out.println("lastKey(): " + treeMap.lastKey());
System.out.println("higherKey(20): " + treeMap.higherKey(20));
System.out.println("lowerKey(20): " + treeMap.lowerKey(20));
System.out.println("ceilingKey(25): " + treeMap.ceilingKey(25));
System.out.println("floorKey(25): " + treeMap.floorKey(25));

// 13. Hashtable - synchronized key-value map
Hashtable<Integer, String> table = new Hashtable<>();
table.put(1, "A");
table.put(2, "B");
System.out.println("\nHashtable: " + table);
System.out.println("get(1): " + table.get(1));
System.out.println("containsKey(2): " + table.containsKey(2));
System.out.println("Size: " + table.size());

System.out.println("\nCollection Framework classes demonstration completed.");

```

```

    }
}

```

5. TERMINAL STYLE / OUTPUT SCREENSHOT

```
C:\Java> javac CollectionClassesDemo.java
C:\Java> java CollectionClassesDemo
```

```
ArrayList: [B, C, A, B]
get(1): C
After set: [B, D, A, B]
indexOf B: 0
Size: 4
```

```
LinkedList: [A, B, C]
getFirst(): A
getLast(): C
poll(): A
After poll: [B, C]
```

```
Vector: [A, B, C]
Size: 3
Capacity: 10
```

```
Stack: [A, B, C]
peek(): C
pop(): C
After pop: [A, B]
```

```
HashSet: [A, B]
Contains A: true
Size: 2
```

```
LinkedHashSet: [C, A, B]
```

```
TreeSet: [10, 20, 30, 40]
first(): 10
last(): 40
higher(20): 30
lower(20): 10
ceiling(25): 30
floor(25): 20
```

```
PriorityQueue: [10, 30, 20]
peek(): 10
poll(): 10
After poll: [20, 30]
```

```
ArrayDeque: [A, B, C]
peekFirst(): A
peekLast(): C
pollFirst(): A
After pollFirst: [B, C]
```

```
HashMap: {101=A, 102=B, 103=C}
get(102): B
containsKey(101): true
Keys: [101, 102, 103]
Values: [A, B, C]
```

```
LinkedHashMap: {101=A, 102=B, 103=C}
entrySet(): [101=A, 102=B, 103=C]
```

```
TreeMap: {10=A, 20=B, 30=C, 40=D}
firstKey(): 10
lastKey(): 40
higherKey(20): 30
lowerKey(20): 10
ceilingKey(25): 30
floorKey(25): 20
```

```
Hashtable: {2=B, 1=A}
get(1): A
containsKey(2): true
Size: 2
```

Collection Framework classes demonstration completed.

6. RESULT

Thus, the important Java Collection Framework classes were successfully studied and implemented. The program demonstrated commonly used methods of list, set, queue, deque and map classes, including insertion, retrieval, removal, searching, sorting and navigation operations.

Note: HashSet, HashMap and Hashtable do not guarantee the same displayed order in every execution. PriorityQueue iteration order also does not represent a fully sorted sequence; its head element has the highest priority according to the queue ordering.